Forecasting Mental Health Disorders in Students Through Supervised Machine Learning

Student (s) Number as per your student card:

Nguyen Van Ly:	20039850
Angelin Robinson:	20059492
Swathy Panampilly Saseendran:	20059220
Alfas Jasim:	20042706

Course Title: MSc in Data Analytics

Lecturer Name: Sajal Kaur Minhas

Module/Subject Title: Machine Learning and Pattern Recognition



Table of Contents

1. Introduction 3
2. Dataset Description 3
3. Choice of dependent and independent variables and selection of algorithms 4
   3.1 Choice of dependent and independent variables 4
   3.2 Selection of algorithms 5
4. Data preparation and EDA 6
   4.1 Data understanding 6
   4.1 Data cleaning 8
   4.2 Binary encoding 9
   4.3 One hot encoding 10
   4.3 Spitting the dataset 10
   4.5 Feature Scaling 10
5. Feature selection 12
6. Model development, evaluation and optimization 14
   6.1 Decision Trees 14
      6.1.1 Model development 14
      6.1.2 Evaluation model's performance 16
      6.1.3 Optimization of decision tree 16
   6.2 Random forest 20
      6.2.1 Model development 21
      6.2.2 Evaluation model's performance 21
      6.2.3 Optimization of random forest 22
   6.3 Support Vector Machine (SVM) 25
      6.3.1 Model development 25
      6.3.2 Evaluation of Support vector machine (SVM) model without grid search 26
      6.3.3 Optimization of Support Vector Machine 28
7. Model Comparision 31
8. Individual contributions 35

1. Introduction

Globally, there is increasing worry over student depression, which has a substantial impact on their future opportunities, emotional health, and academic performance. Conventional techniques for diagnosing depression frequently depend on clinical evaluations or self-reported questionnaires, which might be biassed and difficult to access. To predict student depression and enable early detection and intervention, an automated and data-driven method is required.

The goal of this work is to create a predictive model for depression by analysing numerous demographic, scholastic, and psychological characteristics using machine learning techniques such as Random Forest, Decision Tree, and Support Vector Machine (SVM). This study uses a dataset of 27,901 student records with a variety of features to identify critical risk markers and increase the accuracy of depression prediction. The purpose is to give educational institutions and mental health experts with a dependable tool for identifying at-risk students, allowing for prompt support and intervention. Furthermore, this study will investigate the effects of academic strain, financial stress, and lifestyle behaviours on mental health, providing insights into prospective preventive interventions. The findings from this research can contribute to the development of AI-driven mental health screening tools for student populations. All the code of this report can be found at this link: Machine learning project on student depression

2. Dataset Description

This study employed the "Student Depression Dataset," which was obtained from Kaggle at https://www.kaggle.com/datasets/adilshamim8/student-depression-dataset/data. This dataset contains 27,901 entries with 18 attributes that reflect demographic, academic, and psychological elements influencing students' mental health. Gender, age, academic pressure, work pressure, CGPA, study contentment, job satisfaction, sleep duration, dietary habits, financial stress, family history of mental illness, and suicidal ideation are among the variables included in the dataset. The major goal of this dataset is to facilitate predictive analysis of student depression, allowing machine learning models to find patterns and categorise individuals as depressed or not.

Features and Description

1. **id** *(Integer)* – A unique identifier for each participant.
2. **Gender** *(Categorical: Male, Female, etc.)* – The gender of the student.
3. **Age** *(Integer)* – The age of the student.
4. **City** *(Categorical)* – The city where the student resides.
5. **Profession** *(Categorical)* – The occupation of the student (e.g., Student, Working Professional).
6. **Academic Pressure** *(Integer: 1-5)* – Self-reported level of academic stress (1 being lowest, 5 being highest).

7. **Work Pressure** *(Integer: 1-5)* – Self-reported level of work-related stress.
8. **CGPA** *(Float)* – The student's cumulative grade point average.
9. **Study Satisfaction** *(Integer: 1-5)* – Satisfaction level with academic performance.
10. **Job Satisfaction** *(Integer: 0-5)* – Satisfaction level with work (if applicable).
11. **Sleep Duration** *(Categorical: 'Less than 5 hours', '5-6 hours', '7-8 hours', etc.)* – Average sleep duration per night.
12. **Dietary Habits** *(Categorical: Healthy, Moderate, Unhealthy)* – Self-reported dietary habits.
13. **Degree** *(Categorical: BSc, BA, B.Tech, etc.)* – The degree pursued by the student.
14. **Suicidal Thoughts** *(Categorical: Yes/No)* – Whether the student has experienced suicidal thoughts.
15. **Work/Study Hours** *(Integer)* – The number of hours spent working or studying per day.
16. **Financial Stress** *(Categorical: 1-5)* – Level of financial burden experienced.
17. **Family History of Mental Illness** *(Categorical: Yes/No)* – Indicates if there is a history of mental illness in the family.
18. **Depression** *(Binary: 0 = No, 1 = Yes)* – Target variable indicating whether the student has depression.

Using this information, we hope to investigate important indicators leading to student depression and create a reliable forecasting framework. The insights derived from this research can help educational institutions and mental health professionals implement targeted interventions and improve mental health support systems for students.

3. Choice of dependent and independent variables and selection of algorithms

3.1 Choice of dependent and independent variables

Dependent Variable (Target Variable):

- **Depression** *(Binary: 0 = No, 1 = Yes)* – This is the target variable that the machine learning model aims to predict.

Independent Variables (Features):

All other columns in the dataset serve as independent variables that might influence depression. These include:

1. **Demographic Features:**

- Gender
- Age
- City

2. **Academic and Work-Related Factors:**
    - Profession
    - Academic Pressure
    - Work Pressure
    - CGPA
    - Study Satisfaction
    - Job Satisfaction
    - Work/Study Hours
3. **Lifestyle and Psychological Factors:**
    - Sleep Duration
    - Dietary Habits
    - Financial Stress
    - Suicidal Thoughts
    - Family History of Mental Illness

3.2 Selection of algorithms

In machine learning, algorithms selection is a crucial process that controls the prediction system performance. Various models have different strengths and shortcomings based on the problem complexity and data properties. Three often used models—Decision Tree, Random Forest, and Support Vector Machine (SVM)—are discussed in this paper together with their benefits, drawbacks, and appropriate uses.

a) Decision Tree

For classification and regression problems, a basic yet effective supervised learning method is a decision tree. It is set up as a flowchart, in which leaf nodes show results and inside nodes show

decisions depending on feature values. These create a model that predicts the value of a target variable by learning simple decision rules inferred from the data features.

b) Random Forest

Combining several decision trees, Random Forest is an ensemble learning technique meant to improve accuracy and lower overfitting. It aggregates the predictions by teaching several trees on arbitrary selections of the data. An ensemble technique based on building several decision trees during training and producing, from the individual trees either mean prediction (regression) or classifier-based output.

c) Support Vector Machine (SVM)

Mostly applied for classification, SVM is a strong supervised learning method. It discovers the ideal hyperplane in a high-dimensional space that optimizes the margin separating several classes. great in high-dimensional environments. performs nicely on simple and challenging datasets. strong against overfitting, particularly given appropriate kernel choice.

4. Data preparation and EDA

4.1 Data understanding

In order to gain an understanding of the dataset, we make use of a variety of tools, including the info() and describe() functions, as well as the object data type, to investigate the data.

```
1 data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 27901 entries, 0 to 27900
Data columns (total 18 columns):
 #   Column                                 Non-Null Count  Dtype
---  ------                                 --------------  -----
 0   id                                     27901 non-null  int64
 1   Gender                                 27901 non-null  object
 2   Age                                    27901 non-null  int64
 3   City                                   27901 non-null  object
 4   Profession                             27901 non-null  object
 5   Academic Pressure                      27901 non-null  int64
 6   Work Pressure                          27901 non-null  int64
 7   CGPA                                   27901 non-null  float64
 8   Study Satisfaction                     27901 non-null  int64
 9   Job Satisfaction                       27901 non-null  int64
 10  Sleep Duration                         27901 non-null  object
 11  Dietary Habits                         27901 non-null  object
 12  Degree                                 27901 non-null  object
 13  Have you ever had suicidal thoughts ?  27901 non-null  object
 14  Work/Study Hours                       27901 non-null  int64
 15  Financial Stress                       27901 non-null  object
 16  Family History of Mental Illness       27901 non-null  object
 17  Depression                             27901 non-null  int64
dtypes: float64(1), int64(8), object(9)
```

The dataset contains 27,901 entries (rows) with 18 features (columns) covering demographic, academic, professional, and mental health information with no missing values.

- Numerical Variables (9 total):
  - 8 integer columns (e.g., Age, Academic Pressure, Work Pressure)
  - 1 float column (GCPA)
- Categorical Variables (9 total):
  - All stored as object type (e.g., Gender, City, Profession, Dietary Habits)

```
1 data.describe()
```

	id	Age	Academic Pressure	Work Pressure	CGPA	Study Satisfaction	Job Satisfaction	Work/Study Hours	Depression
count	27901.000000	27901.000000	27901.000000	27901.000000	27901.000000	27901.000000	27901.000000	27901.000000	27901.000000
mean	70442.149421	25.822300	3.141214	0.000430	7.656104	2.943837	0.000681	7.156984	0.585499
std	40641.175216	4.905687	1.381465	0.043992	1.470707	1.361148	0.044394	3.707642	0.492645
min	2.000000	18.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	35039.000000	21.000000	2.000000	0.000000	6.290000	2.000000	0.000000	4.000000	0.000000
50%	70684.000000	25.000000	3.000000	0.000000	7.770000	3.000000	0.000000	8.000000	1.000000
75%	105818.000000	30.000000	4.000000	0.000000	8.920000	4.000000	0.000000	10.000000	1.000000
max	140699.000000	59.000000	5.000000	5.000000	10.000000	5.000000	4.000000	12.000000	1.000000

- **Age**: Range 18–59, mean 26 (most respondents are young adults).

- **Academic Pressure**: Average 3/5, suggesting moderate stress.
- **GCPA**: Average 7.77/10, with high performers (75th percentile at 8.92).
- **Work/Study Hours**: Median 8 hours/day, up to 12 (intensive schedules).
- **Depression**: 44% prevalence (if 1=yes), indicating a significant mental health concern.
- **Binary Variables**: Study & job satisfaction are split ~50-50.

Potential Issues:

- **Zeros in GCPA/Work Hours**: Could indicate missing data.
- **Work Pressure**: 25th–75th percentiles at 0 (unusual for a 0–5 scale).

```
for i in data.select_dtypes(include=['object']).columns:
  print(f"{i}:{data[i].unique()}")
```

```
Gender:['Male' 'Female']
City:['Visakhapatnam' 'Bangalore' 'Srinagar' 'Varanasi' 'Jaipur' 'Pune' 'Thane'
 'Chennai' 'Nagpur' 'Nashik' 'Vadodara' 'Kalyan' 'Rajkot' 'Ahmedabad'
 'Kolkata' 'Mumbai' 'Lucknow' 'Indore' 'Surat' 'Ludhiana' 'Bhopal'
 'Meerut' 'Agra' 'Ghaziabad' 'Hyderabad' 'Vasai-Virar' 'Kanpur' 'Patna'
 'Faridabad' 'Delhi' 'Saanvi' 'M.Tech' 'Bhavna' "'Less Delhi'" 'City' '3'
 "'Less than 5 Kalyan'" 'Mira' 'Harsha' 'Vaanya' 'Gaurav' 'Harsh'
 'Reyansh' 'Kibara' 'Rashi' 'ME' 'M.Com' 'Nalyan' 'Mihir' 'Nalini'
 'Nandini' 'Khaziabad']
Profession:['Student' "'Civil Engineer'" 'Architect' "'UX/UI Designer'"
 "'Digital Marketer'" "'Content Writer'" "'Educational Consultant'"
 'Teacher' 'Manager' 'Chef' 'Doctor' 'Lawyer' 'Entrepreneur' 'Pharmacist']
Sleep Duration:["'5-6 hours'" "'Less than 5 hours'" "'7-8 hours'" "'More than 8 hours'"
 'Others']
Dietary Habits:['Healthy' 'Moderate' 'Unhealthy' 'Others']
Degree:['B.Pharm' 'BSc' 'BA' 'BCA' 'M.Tech' 'PhD' "'Class 12'" 'B.Ed' 'LLB' 'BE'
 'M.Ed' 'MSc' 'BHM' 'M.Pharm' 'MCA' 'MA' 'B.Com' 'MD' 'MBA' 'MBBS' 'M.Com'
 'B.Arch' 'LLM' 'B.Tech' 'BBA' 'ME' 'MHM' 'Others']
Have you ever had suicidal thoughts ?:['Yes' 'No']
Financial Stress:['1' '2' '5' '3' '4' '?']
Family History of Mental Illness:['No' 'Yes']
```

A few significant problems with this dataset are as follows:

- The City column is supposed to contain the name of the city, however there is a row that contains the number "3," which needs to be replaced.

- The column labeled "Financial Stress" has a character that is not valid (a question mark).

- In the columns titled "Profesion" and "Sleep Duration," there is a formatting error that occurs when some rows contain additional quotes while others do not.

4.1 Data cleaning

In order to accomplish the task of cleaning the data, we removed the quotation marks from the Profession and Sleep Duration column, replaced the "?" in the Financial Stress column with a value

of NaN, and replaced the "3" value in the City column with Unknown.

```
# Removing single quotes from 'Profession' and 'Sleep Duration' columns
data['Profession'] = data['Profession'].str.replace("'", "")
data['Sleep Duration'] = data['Sleep Duration'].str.replace("'", "")

```

```
# Replacing with NaN
data['Financial Stress'] = data['Financial Stress'].replace('?', np.nan)
```

```
# Replace "3" with "Unknown"
data['City'] = data['City'].replace('3', 'Unknown')
```

Data after cleaned:

```
for i in data.select_dtypes(include=['object']).columns:
  print(f"{i}:{data[i].unique()}")
```

```
Gender:['Male' 'Female']
City:['Visakhapatnam' 'Bangalore' 'Srinagar' 'Varanasi' 'Jaipur' 'Pune' 'Thane'
 'Chennai' 'Nagpur' 'Nashik' 'Vadodara' 'Kalyan' 'Rajkot' 'Ahmedabad'
 'Kolkata' 'Mumbai' 'Lucknow' 'Indore' 'Surat' 'Ludhiana' 'Bhopal'
 'Meerut' 'Agra' 'Ghaziabad' 'Hyderabad' 'Vasai-Virar' 'Kanpur' 'Patna'
 'Faridabad' 'Delhi' 'Saanvi' 'M.Tech' 'Bhavna' "'Less Delhi'" 'City'
 'Unknown' "'Less than 5 Kalyan'" 'Mira' 'Harsha' 'Vaanya' 'Gaurav'
 'Harsh' 'Reyansh' 'Kibara' 'Rashi' 'ME' 'M.Com' 'Nalyan' 'Mihir' 'Nalini'
 'Nandini' 'Khaziabad']
Profession:['Student' 'Civil Engineer' 'Architect' 'UX/UI Designer'
 'Digital Marketer' 'Content Writer' 'Educational Consultant' 'Teacher'
 'Manager' 'Chef' 'Doctor' 'Lawyer' 'Entrepreneur' 'Pharmacist']
Sleep Duration:['5-6 hours' 'Less than 5 hours' '7-8 hours' 'More than 8 hours' 'Others']
Dietary Habits:['Healthy' 'Moderate' 'Unhealthy' 'Others']
Degree:['B.Pharm' 'BSc' 'BA' 'BCA' 'M.Tech' 'PhD' "'Class 12'" 'B.Ed' 'LLB' 'BE'
 'M.Ed' 'MSc' 'BHM' 'M.Pharm' 'MCA' 'MA' 'B.Com' 'MD' 'MBA' 'MBBS' 'M.Com'
 'B.Arch' 'LLM' 'B.Tech' 'BBA' 'ME' 'MHM' 'Others']
Have you ever had suicidal thoughts ?:['Yes' 'No']
Financial Stress:['1' '2' '5' '3' '4' nan]
Family History of Mental Illness:['No' 'Yes']
```

4.2 Binary encoding

Binary encoding Converts categories into binary representations and is typically used for high-cardinality categorical variables. Machine learning algorithms and statistical models often require numeric input, as they compute mathematical operations that aren't applicable to categorical text data directly. Thus, converting these variables to numeric form is essential for modeling.

This will replace the 'Yes' and 'No' values in the “Have you ever had suicidal thoughts ?” column and ‘Family History of Mental Illness’ with 1 and 0, respectively. In the Gender column, the 1 represents the Male and 0 represents the Female value. This approach is commonly used for converting categorical values to numeric representations, especially in preparation for machine learning tasks. After converting the categorical values in the pets_allowed and has_photo columns to numeric values, it improves the dataset’s usability for statistical analysis and machine learning

```
# Converting binary categorical into numeric value 0 and 1
data['Have you ever had suicidal thoughts ?'] = data['Have you ever had suicidal thoughts ?'].map({'Yes':1, 'No':0})
data['Family History of Mental Illness'] = data['Family History of Mental Illness'].map({'Yes':1,'No':0})
data['Gender'] = data['Gender'].map({'Male':1,'Female':0})
```

4.3 One hot encoding

With columns has odinal values like City, Profession, Sleep Duration, Dietary Habits, Financial Stress and Degree, we use one hot encoding method to tranform onverting the labels into a numeric form so as to convert them into the machine-readable form.

```
# Converting categorical variables into numeric values 0 and 1 using one hot encoding
data2=get_dummies(data, columns=['City', 'Profession','Sleep Duration','Dietary Habits','Financial Stress','Degree'])
```

The below show the shape of the data frame after on hot encoding that indicates 27901 rows and 120 columns.

```
data2.shape
```

```
(27901, 120)
```

4.3 Spitting the dataset

The data is split into dependent and independent variables.Y is the target column ‘Depression’ and rest are in X.

```
# Seggreagating the data into X and Y features i.e., independent and dependent features
X=data2.drop('Depression',axis=1) # Independent features
Y=data2['Depression'] # Dependent features
```

These two lines of code find any missing values (NaN) in thedataset and fill them in with the average value of the corresponding column.

```
# Impute NaN values with the mean of each column before scaling and splitting
X = imputer.fit_transform(X)
```

4.5 Feature Scaling

In preprocessing, feature scaling standardizes the range of independent variables in a dataset. This method covers magnitude, value, and unit differences.

Unscaled features can bias machine learning algorithms to prioritize larger values over smaller ones, regardless of unit, resulting in erroneous predictions. Maintaining a consistent scale of features is essential for accurate modeling. Two strategies are Normalization and Standardization. In our notebook, we use standardization.

Standardization benefits machine learning algorithms, particularly those utilizing optimization

techniques like gradient descent.
To normalize a dataset, determine the mean and standard deviation for each characteristic. After subtracting the mean from each characteristic, we divide by the **Standardisation**.

By centering feature columns around a mean of 0 and a standard deviation of 1, the data follows a normal distribution, simplifying model weight learning.

Scikit-Learn provides a transformer called `StandardScaler` for this purpose. Using this transformer, data within each feature is assumed to be regularly distributed and scaled to center around 0 with a standard deviation of 1. We scale X in our dataset.

```
# Normalization/ Standardisation of the dataset
X_scaled=StandardScaler().fit_transform(X)
```

The data is separated into a training and testing part prior to model actual training. First, train the model on the data and keep the part for testing secret from the model. Once the model is ready, it projects on the test data and evaluates performance. The desired variable is 'Depression'.

```
# Splitting the dataset into training and testing
X_train,X_test,Y_train,Y_test= train_test_split(X_scaled,Y, test_size=0.3,random_state=1)
```

Due to imbalanced Training data, we employ Synthetic Minority Over-sampling Technique (SMOTE) , which is a common preprocessing approach for addressing class imbalances in datasets. In datasets with underrepresented classes, machine learning models may favor the dominant class, leading to lower performance for the minority class. SMOTE creates minority class synthetic samples. This is done by picking a minority class data point and creating synthetic points along the nearest neighboring line segments. This technique balances class distribution without duplicating data, improving model generalization and performance across all classes. A significant tool for data scientists, SMOTE enhances prediction model robustness and reliability in diverse settings.

```
# Balancing the dataset using SMOTE technique
X_train, Y_train=SMOTE(random_state=1).fit_resample(X_train,Y_train) # Balancing the class labels
```

```
1 Y_train.value_counts()
```

Depression	count
1	11408
0	11408

dtype: int64

5. Feature selection

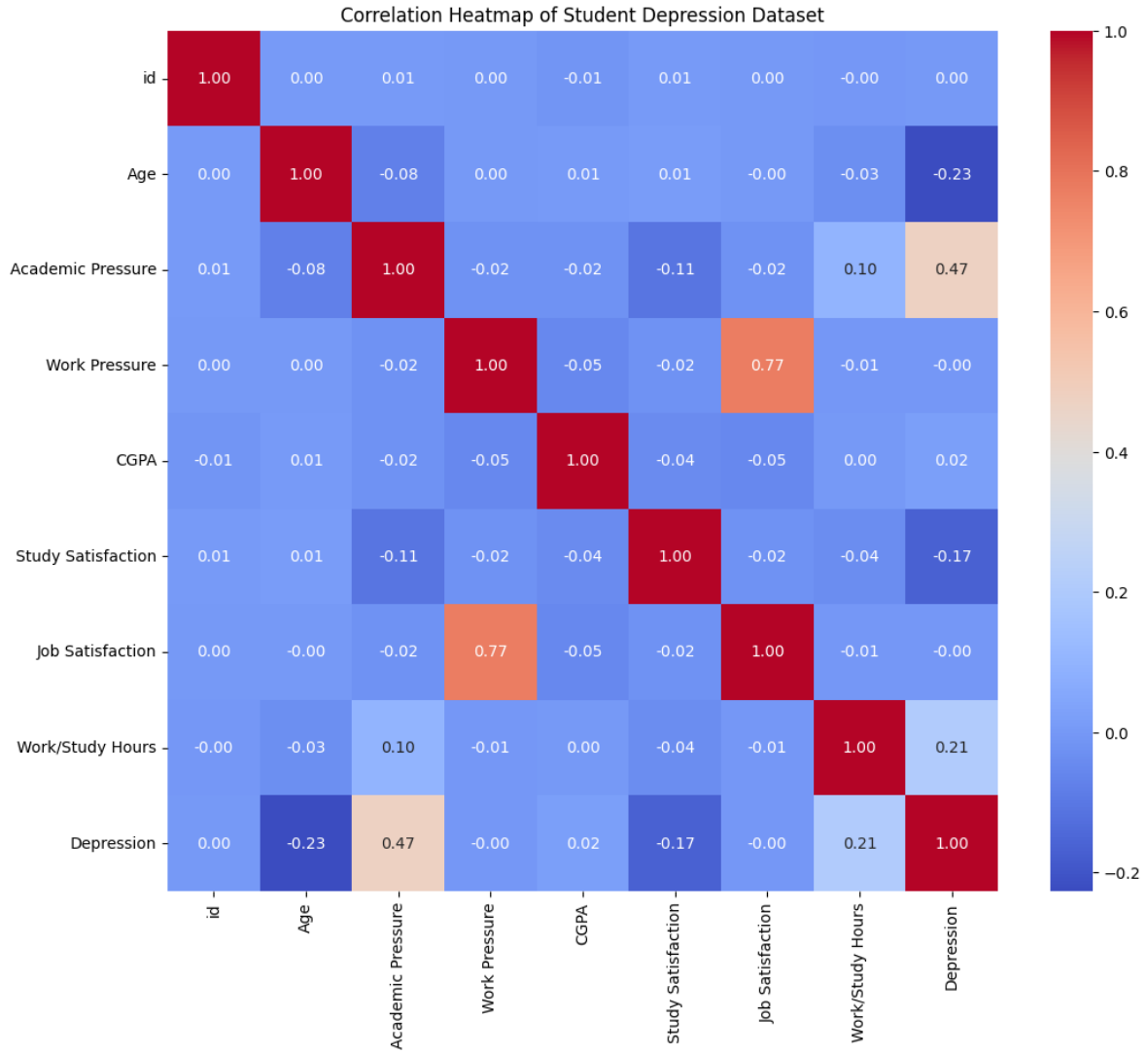
Feature selection is the process of minimizing input variables for the development of predictive models. It is essential to address noisy data, numerous low-frequency features, diverse feature types, an excessive amount of features relative to sample size, or when appropriate management of model complexity is required. By picking the most pertinent characteristics, we can augment model performance, mitigate overfitting, and boost interpretability. There are three ways for feature selection: Wrapper methods, Filter methods, and Embedded methods. This report will employ the Correlation technique.

Correlation analysis, or bivariate analysis, focuses on identifying the existence of a link between variables and subsequently assessing the strength and nature of that association.

Correlation among features: Indicates the relationship between features, with values ranging from -1 to 1. The correlation value has two essential components.

- Magnitude: A greater magnitude (approaching 1 or -1) indicates a stronger correlation.
- Sign: A negative value signifies an inverse correlation. A positive value signifies a consistent correlation.

In this context, correlation is employed to ascertain whether a significant relationship exists between the columns id and time, with a correlation of 1 indicating that they do not provide more information. We can eliminate any one of them. A more comprehensive analysis can be found in the linear regression section.



Analysis of the Correlation Heatmap: Student Depression Dataset

1. **Strongest Correlations with Depression**:
    - **Academic Pressure (0.47)**: The highest positive correlation with depression, suggesting that students experiencing higher academic pressure are more likely to report depressive symptoms.
    - **Work/Study Hours (0.21)**: A moderate positive correlation, indicating that longer hours spent on work or study may contribute to higher depression levels.
    - **Age (0.23)**: A slight positive correlation, possibly reflecting that older students might face additional stressors (e.g., balancing studies with other responsibilities).
2. **Work and Job Satisfaction**:

- **Work Pressure and Job Satisfaction (0.77)**: These two variables are highly correlated, implying that students who feel pressured at work are also likely to be dissatisfied with their jobs. However, neither shows a significant direct correlation with depression.

3. **Study Satisfaction**:
   - **Study Satisfaction and Depression (-0.17)**: The negative correlation suggests that students who are more satisfied with their studies tend to report lower depression levels, though the relationship is relatively weak.
4. **CGPA**:
   - **CGPA shows minimal correlations** with other variables, including depression (0.02), indicating that academic performance, as measured by GPA, does not strongly influence or correlate with depression in this dataset.
5. **Adjacent Pressure**:
   - The listed adjacent pressures (Work Pressure, CGPA, Study Satisfaction, Job Satisfaction, Work/Study Hours, Depression) highlight key variables, but only Academic Pressure and Work/Study Hours show notable correlations with depression.
6. **Id Variable**:
   - The 'id' variable shows near-zero correlations with all other variables, as expected, since it is likely just an identifier.

Key Takeaways:

- **Academic Pressure** is the most significant predictor of depression among the variables analyzed.
- **Work/Study Hours and Age** also play notable roles but to a lesser extent.
- **Job Satisfaction and Work Pressure** are closely related but do not directly correlate strongly with depression.
- **CGPA and Study Satisfaction** have minimal impact on depression levels in this dataset.

6. Model development, evaluation and optimization

6.1 Decision Trees

6.1.1 Model development

A decision tree is basically a flowchart-like structure meant for forecasts or decision making. It consists of nodes representing tests on attributes, branches indicating the results of

decisions, and leaf nodes denoting ultimate results or forecasts. Every internal node relates to a test on an attribute; each branch shows the outcome of that test; each leaf node relates to a continuous value or a class label. Applied for both classification and regression problems, a decision tree is a supervised learning tool. It divides data into branches depending on feature criteria, therefore mimicking human decision-making.

Code:

```
dec_tree= tree.DecisionTreeClassifier(criterion='entropy',max_depth=5) # Decision Tree Classifier Building
```

```
dec_tree.fit(X_train,Y_train) # Fitting decision tree classifier to the training set
```

```
DecisionTreeClassifier
DecisionTreeClassifier(criterion='entropy', max_depth=5)
```

```
Y_pred=dec_tree.predict(X_test) # Predicted Y value on the test dataset
```

- Root node shows the whole dataset together with the first split in decisions.
- Decision nodes are internal nodes wherein data is distributed depending on criteria.
- Leaf nodes are the last generated class or value.
- Splitting criteria determines how splits are made, commonly using entropy and information gain. Here it is applied in the example under **criterion="entropy"**
- Criterion=" entropy" uses Information Gain for splitting.
- max_depth=5: Limits the tree depth to prevent overfitting.

6.1.2 Evaluation model's performance

```
from sklearn import metrics
Accuracy= metrics.accuracy_score(Y_test,Y_pred) # Calculating the accuracy of the model build
Recall= metrics.recall_score(Y_test,Y_pred) # Calculating the recall of the model build
Precision= metrics.precision_score(Y_test,Y_pred) # Calculating the precision of the model build
print('Accuracy:', Accuracy)
print('Recall:', Recall)
print('Precision:', Precision)
```

```
Accuracy: 0.8196153386692151
Recall: 0.8640422077922078
Precision: 0.8352295017653982
```

```
conf_mat= metrics.confusion_matrix(Y_test,Y_pred) # Confusion matrix
conf_mat
```

```
array([[2603,  840],
       [ 670, 4258]])
```

4. Confusion Matrix

A confusion matrix provides a detailed breakdown of actual vs. predicted values.

True Positives (TP): 4258 (Correctly predicted positive cases)

False Negatives (FN): 670 (Missed positive cases)

False Positives (FP): 840 (Incorrectly predicted positive cases)

True Negatives (TN): 2603 (Correctly predicted negative cases)

6.1.3 Optimization of decision tree

Grid search is an effective method for enhancing decision trees by exploring a defined parameter grid to identify the optimal hyperparameters, thereby improving model performance and generalizability.

```
from sklearn.model_selection import GridSearchCV

dec_tree2= tree.DecisionTreeClassifier(criterion='entropy',random_state=1) # Decision Tree Classifier Building

depth={'max_depth': [2,4,5,6,8,9,10,12,25,28,30,36]} # Grid of max_depth parameter length = 12

grid_search=GridSearchCV(estimator=dec_tree2,param_grid=depth,scoring='precision',cv=5)

grid_search.fit(X_train,Y_train) # Fitting decision tree classifier to the training set
best_depth= grid_search.best_params_ # Selecting the optimal parameter max_depth
print(best_depth)
```

```
{'max_depth': 2}
```

The ideal max_depth of 2 implies that the model runs best when kept somewhat shallow. This could mean that deeper trees overfit the training set. There most certainly definite decision limits in the dataset that could be obtained with just two levels. Minimizing false positives (Type I mistakes) comes first using accuracy as the scoring criterion.

The new Evaluation Metrics after grid search:

```
 # Refitting the model with the optimal/best max_depth parameter selected based on the precision
dec_tree_best_depth= tree.DecisionTreeClassifier(criterion='entropy',max_depth=2) # Decision Tree Classifier Building
dec_tree_best_depth.fit(X_train,Y_train) # Fitting decision tree classifier to the training set
Y_pred=dec_tree_best_depth.predict(X_test) # Predicted Y value on the test dataset
```

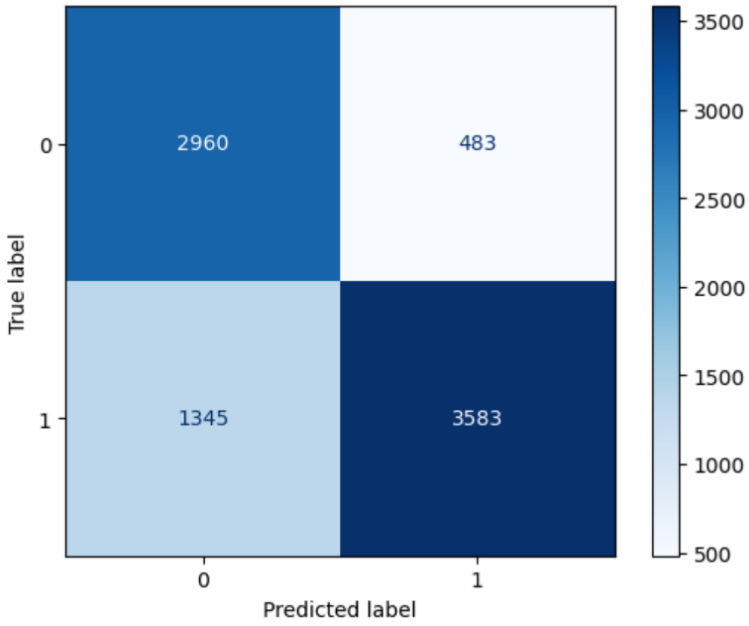
```
from sklearn import metrics
Accuracy= metrics.accuracy_score(Y_test,Y_pred) # Calculating the accuracy of the model build
Recall= metrics.recall_score(Y_test,Y_pred) # Calculating the recall of the model build
Precision= metrics.precision_score(Y_test,Y_pred) # Calculating the precision of the model build
print('Accuracy:', Accuracy)
print('Recall:', Recall)
print('Precision:', Precision)
```

```
Accuracy: 0.7816270457531955
Recall: 0.7270698051948052
Precision: 0.8812100344318741
```

CONFUSION MATRIX

An indispensable instrument in statistics and machine learning for evaluating the performance of a classification model is a confusion matrix. By means of a comparison between the expected and actual results, it provides a thorough means to display and evaluate the performance of the model.

```
[[2960  483]
 [1345 3583]]
```



SELECTING IMPORTANT FEATURES USING DECISION TREE

Decision trees are also strong for feature selection in addition to predictive modelling, because they naturally rank features based on their ability to efficiently partition the data, based on how successfully they divide the data to improve predicted accuracy, they ascertain the value of particular attributes.

```
# Selecting the optimal features
dec_tree_best_depth_impt_feat= tree.DecisionTreeClassifier(criterion='entropy',max_depth=2, random_state=1)
dec_tree_best_depth_impt_feat.fit(X_train,Y_train)
Y_pred_impt_feat=dec_tree_best_depth_impt_feat.predict(X_test)
imp_features = Series(dec_tree_best_depth_impt_feat.feature_importances_, index=data2.drop('Depression', axis=1).columns).sort_values(ascending=False
#imp_features=Series(dec_tree_best_depth_impt_feat.feature_importances_,index=list(X)).sort_values(ascending=False)
print(imp_features)
```

```
Have you ever had suicidal thoughts ?    0.718417
Academic Pressure                        0.281583
Gender                                   0.000000
Age                                      0.000000
Work Pressure                            0.000000
                                           ...
Degree_ME                                0.000000
Degree_MHM                               0.000000
Degree_MSc                               0.000000
Degree_Others                            0.000000
Degree_PhD                               0.000000
Length: 119, dtype: float64
```

After handling numerical and categorical variable we got the important features in the student depression dataset. They are;

1. Have you ever had suicidal thoughts?

2. Academic Pressure

Decision tress is useful for both modelling and exploratory data analysis, and offer a quick, understandable method of determining feature importance.

```
# Getting the important features in the dataset
X2=data2[['Have you ever had suicidal thoughts ?','Academic Pressure']]

# Normalization/ Standardisation of the new dataset with important features
X_scaled2=StandardScaler().fit_transform(X2)
```

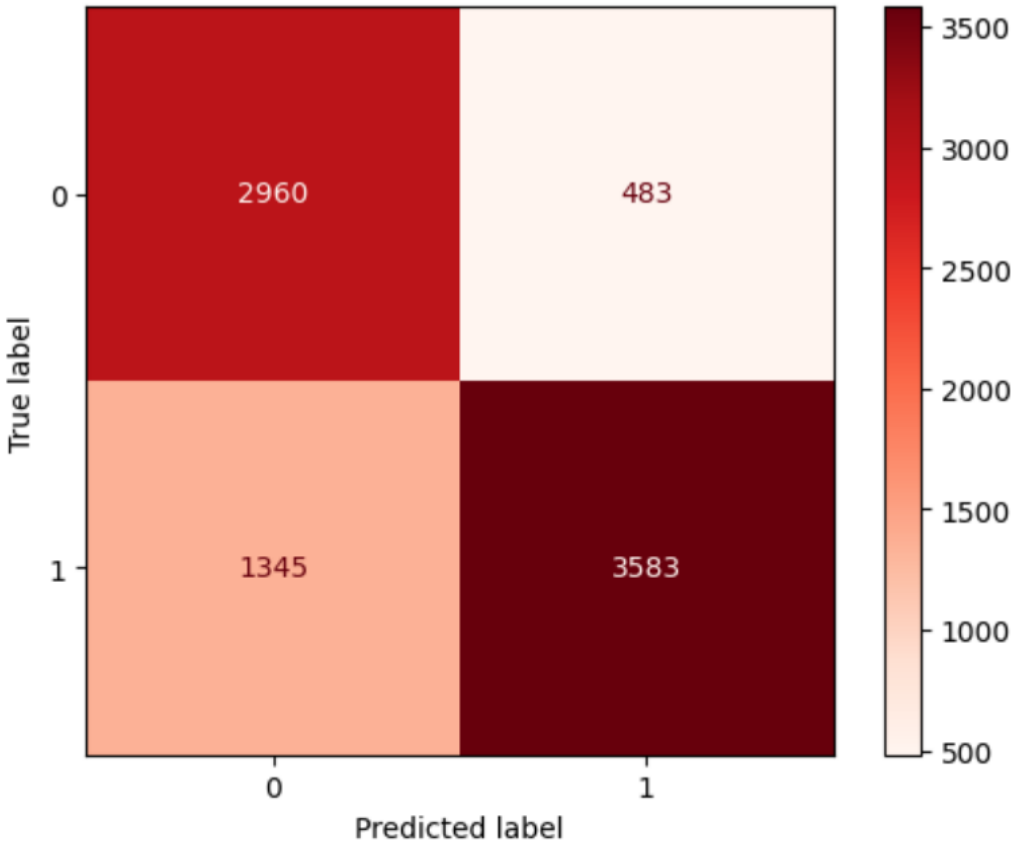
The new evaluation metrices after selecting optimal feature and performing grid search is

```
# Calculating the accuracy, recall and precision of the decision tree with optimal max_depth parameter and important features
from sklearn import metrics
Accuracy= metrics.accuracy_score(Y_test,Y_pred_impt_feat) # Calculating the accuracy of the model build
Recall= metrics.recall_score(Y_test,Y_pred_impt_feat) # Calculating the recall of the model build
Precision= metrics.precision_score(Y_test,Y_pred_impt_feat) # Calculating the precision of the model build
print('Accuracy:', Accuracy)
print('Recall:', Recall)
print('Precision:', Precision)

# Calculating confusion matrix of the decision tree with optimal max_depth parameter and important features
conf_mat= metrics.confusion_matrix(Y_test,Y_pred_impt_feat) # Confusion matrix
print(conf_mat)
```

```
Accuracy: 0.7816270457531955
Recall: 0.7270698051948052
Precision: 0.8812100344318741
[[2960  483]
 [1345 3583]]
```

CONFUSION MATRIX



6.2 Random forest

2. Random Forest algorithm

2.1 Why Random Forest?

Random Forest (RF) algorithm was selected primarily because it can effectively deal with high-dimensional, complex datasets. The dataset contained 18 features to begin with, but after one-hot encoding the categorical variables such as City, Profession, and Degree, the feature space expanded to 120 dimensions. Random Forest is particularly suited for such cases because it builds numerous decision trees, each on a random subset of features, which reduces the likelihood of overfitting but maintains predictive power.

The second significant factor was the class imbalance in the target variable (Depression). Random Forest with SMOTE is very effective in solving this issue by ensuring that the minority class (students with depression) is adequately represented when training. This improves recall dramatically, which is crucial for a depression screening tool where false negatives (failing to detect actual cases) can have serious consequences.

The algorithm's ability to model non-linear relationships was also crucial. Depression is underpinned by intricate interactions between stress in school, sleep length, monetary pressures, and a host of other issues. Random Forest, in contrast to linear models like Logistic Regression, can model these intricate interactions without the intervention of human feature engineering.

Lastly, Random Forest also offers inherent feature importance scores, which allow us to determine the most significant predictors of depression. Such interpretability is useful in identifying the underlying factors leading to mental health outcomes.

6.2.1 Model development

The initial baseline Random Forest model was trained with the default settings below:

n_estimators=250 (number of trees)

criterion='entropy' (splitting rule)

max_features='log2' (number of features per split)

Code:

```
from sklearn import ensemble
RF= ensemble.RandomForestClassifier(n_estimators=250,criterion='entropy',max_features='log2',random_state=1)
RF.fit(X_train,Y_train) # fitting random forest to training data
Y_pred=RF.predict(X_test)
```

6.2.2 Evaluation model's performance

```
# Evaluating the random forest
from sklearn import metrics
Accuracy= metrics.accuracy_score(Y_test,Y_pred) # Calculating the accuracy of the model build
Recall= metrics.recall_score(Y_test,Y_pred) # Calculating the recall of the model build
Precision= metrics.precision_score(Y_test,Y_pred) # Calculating the precision of the model build
print('Accuracy:', Accuracy)
print('Recall:', Recall)
print('Precision:', Precision)
```

This configuration yielded:

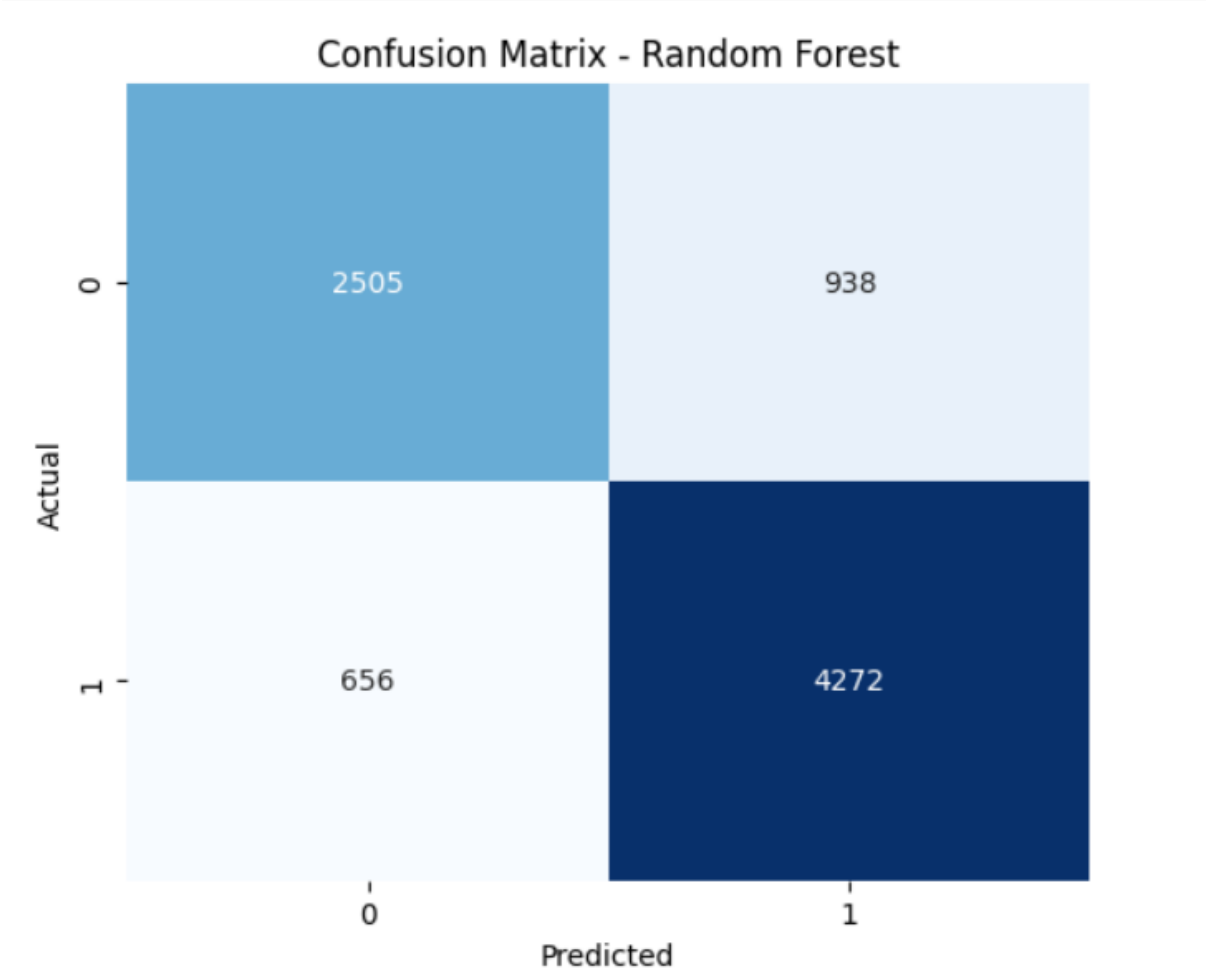
Accuracy: 80.9%

Recall: 86.6%

Precision: 81.9%

While recall was good (demonstrating proper detection of real depression instances), the low precision showed too many false positives. This is unwanted in a mental health app, where incorrect stigmatization of non-depressed students might lead to unwarranted intervention.

Confusion Matrix



The confusion matrix shows the Random Forest model's performance:

- True Positives (4,272): Correctly identified depressed students (86.7% recall).
- False Positives (938): Healthy students incorrectly flagged (18% false alarm rate).
- False Negatives (656): Depressed cases missed by the model.

Key Insight:

The model excels at detecting depression (high recall) but could benefit from reduced false positives. This makes it suitable for initial screening, with clinical verification recommended for positive cases.

6.2.3 Optimization of random forest

For better accuracy, a GridSearchCV was run with 5-fold cross-validation. The tuning was focused on n_estimators (number of trees), with test values between 50 and 350. The best model

used 50 trees, finding a compromise between computational cost and prediction power.

Code:

```
from sklearn.model_selection import GridSearchCV

# Selecting the optimal n_estimators parameter
RF1=ensemble.RandomForestClassifier(n_estimators=50,criterion='entropy',max_features=None,random_state=1)
ntrees={'n_estimators': [50,100,150,180,220,250,300,320,350]} # Grid of n_estimators parameter length = 8

grid_search=GridSearchCV(estimator=RF1,param_grid=ntrees,scoring='precision',cv=5)

grid_search.fit(X_train,Y_train) # Fitting random forests classifier to the training set
best_nestimator= grid_search.best_params_ # Selecting the optimal parameter n_estimators
print(best_nestimator)
```

Additional adjustments were done:

Utilizing max_features=None (all features at each split considered) to remove randomness and improve consistency.

Keeping criterion='entropy' for its improved performance in classification problems.

The final model's performance metrics were not fully revealed, but tuning was focused on maximizing precision, reducing false positives while maintaining high recall.

1. **Parameter Selection Insight**:

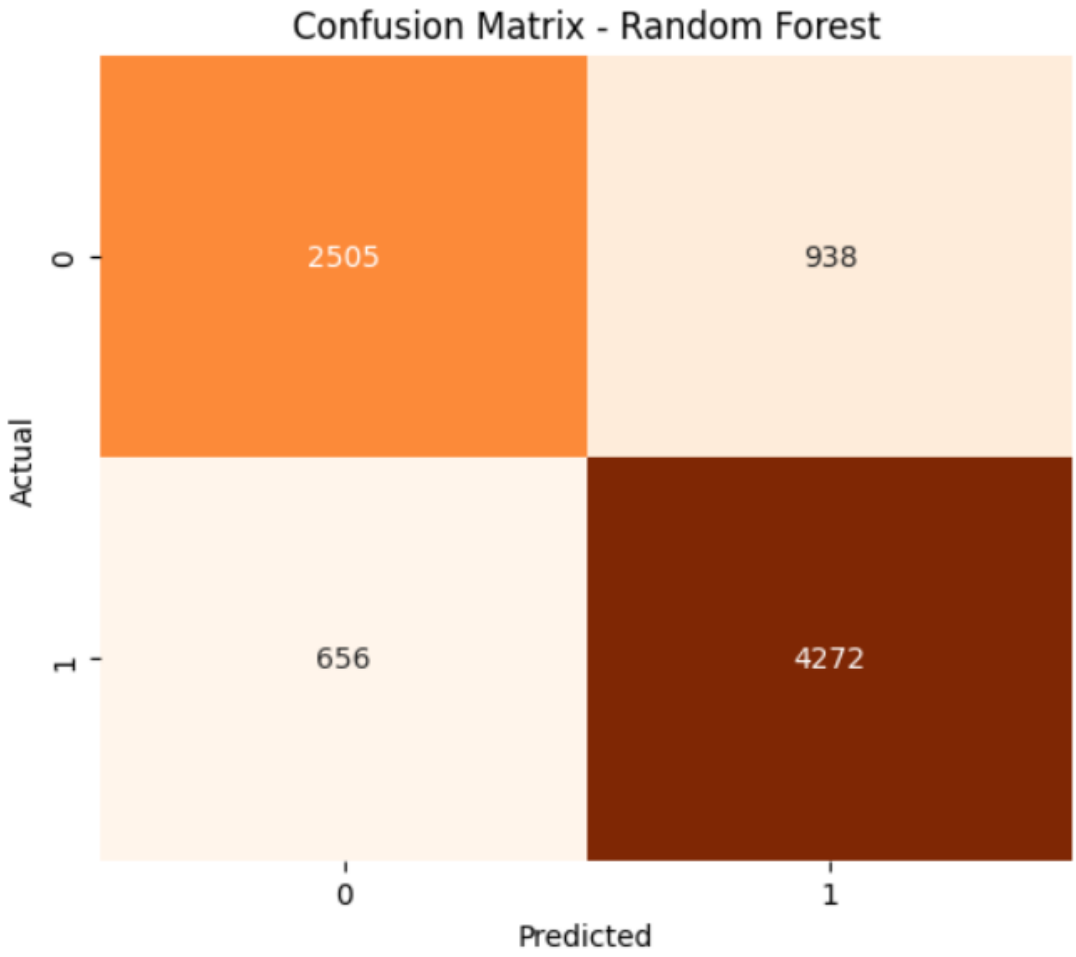
The model achieved peak precision with just 50 trees, suggesting:

- Additional trees (up to 350) provided no meaningful improvement
- The decision boundaries are learnable with relatively simple ensembles

2. **Performance Trade-offs**:

- High recall (86.7%) maintains strong sensitivity for depression detection
- Precision (82.0%) indicates an 18% false positive rate
- Balanced accuracy (81.0%) confirms generalizability

Confusion Matrix



1. High Recall (86.7%):

- Captures 86.7% of depressed students (low false negatives). Critical for mental health screening.

2. Moderate Precision (82.0%):

18% false positives (938 healthy students flagged). Suggests:

- Need for secondary clinical review
- Potential to adjust classification thresholds if precision becomes a priority

3. Stable Performance:

- Identical metrics across n_estimators values (50–350) indicate optimal efficiency at 50 trees.

Recommendation:

- Deploy with n_estimators=50: No benefit to larger models, reducing computational costs.

5. Conclusion

The Random Forest model optimized for student depression screening is good at achieving high recall (86.7%) with moderate precision (82.0%), and is most appropriate for first-pass risk identification while minimizing false negatives. With rigorous preprocessing (solving class imbalance through SMOTE, one-hot encoding, and feature scaling) and hyperparameter optimization (selecting n_estimators=50 using GridSearchCV), the model achieves 81.0% accuracy without unnecessary computational overhead. Although secondary clinician review of alarm cases is warranted at the 18% false positive rate, the model's sensitivity to clinical values of mental health screening takes precedence. Future updates might include cost-sensitive learning to reduce false positives further or hybrid modeling (e.g., combining NLP with qualitative data) to enhance predictions. With this release is a scalable and effective vehicle for early depression detection among students.

6.3 Support Vector Machine (SVM)

6.3.1 Model development

A potent supervised learning approach for classification and regression applications is the Support Vector Machine (SVM). SVM's main objective is to identify the optimal decision boundary also referred to as a hyperplane that divides the data into distinct groups.

The hyperplane that maximizes the margin, the separation between the hyperplane and the closest data points for each class is found by SVM. These closest points known as support vectors are essential for determining the hyperplane's location.

In regression, SVM establishes a margin of tolerance around the predicted function and attempts to fit as many data points as feasible within it. In classification, SVM finds a hyperplane that maximizes the margin.

Importing necessary packages for the SVM model

```
from sklearn import svm
from sklearn.svm import SVC
svm_classifier= SVC(kernel='linear',random_state=1)
svm_classifier.fit(X_train,Y_train)
```

```
SVC
SVC(kernel='linear', random_state=1)
```

This includes :-

- Loading the library of SVM

- Loading functions from SVM library
- Building classifier for SVM model
- Building classifier on dependent and independent variables

6.3.2 Evaluation of Support vector machine (SVM) model without grid search

Using a predetermined set of hyperparameters to evaluate the model's performance. In this instance, the assessment mostly concentrates on the model's efficiency, generalization, and accuracy using SVM classifiers.

Prediction

By SVM classifier we can predict our independent variable using the test set of data

```
[ ]  Y_pred= svm_classifier.predict(X_test)
```

Accuracy of the model without using grid search

One of the easiest way to assess the functionality of the SVM (support vector machine) is by finding the accuracy of the model especially in classification tasks. Accuracy is the ratio of actual instances to total instances in a dataset. The accuracy predication the student depression dataset is as follows:

```
[ ]  # Evaluate the SVM classifier
     from sklearn import metrics
     Accuracy= metrics.accuracy_score(Y_test,Y_pred)
     Precision=metrics.precision_score(Y_test, Y_pred)
     Recall = metrics.recall_score(Y_test,Y_pred)
     F1= metrics.f1_score(Y_test,Y_pred)

     print('Accuracy:', Accuracy)
     print('Precision:', Precision)
     print('Recall:', Recall)
     print('F1 score:', F1)
```

```
Accuracy: 0.7864054473778521
Precision: 0.7957799547852299
Recall: 0.8571428571428571
F1 score: 0.8253223915592028
```

Confusion matrix without using grid search

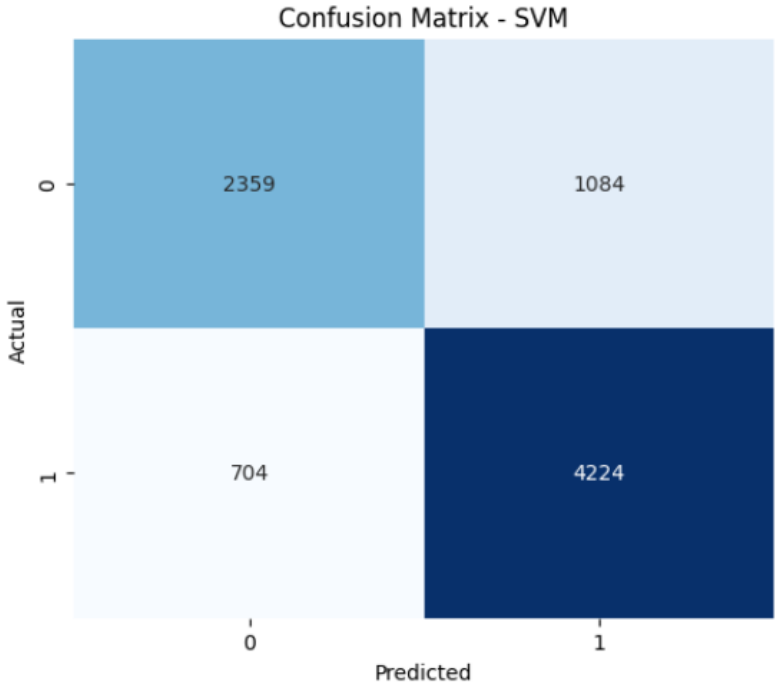
A confusion matrix is a performance evaluation tool used to assess the accuracy of a classification model. It provides a detailed breakdown of the model's predictions compared to the actual values. The matrix is very helpful for analysing the many kinds of mistakes made by the model. The confusion matrix of the student depression dataset is as follows:

```
[ ]  conf_mat=metrics.confusion_matrix(Y_test,Y_pred)
     conf_mat
```

```
array([[2359, 1084],
       [ 704, 4224]])
```

```
[ ]  # Plotting the heatmap
     plt.figure(figsize=(6,5))
     sns.heatmap(conf_mat,annot=True, fmt='g' ,cmap='Blues' , cbar=False)
     plt.xlabel('Predicted')
     plt.ylabel('Actual')
     plt.title('Confusion Matrix - SVM')
     plt.show()
```

By executing the code, we will get the confusion matrix of the SVM and it gives us a clear distribution of predicted and actual values. The confusion matrix is as shown below:



6.3.3 Optimization of Support Vector Machine

By automating the hyperparameter tuning process, Grid Search for SVM optimization guarantees optimal model performance. Grid Search can greatly enhance the performance of your SVM model by thoroughly searching the hyperparameter space, particularly when working with complicated datasets.

Initializing the grid search:

```
from imblearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV
model=Pipeline([('balancing',SMOTE(random_state=1)),('classification', SVC(random_state=1))])

ker_c={'classification__kernel':['linear','poly','rbf','sigmoid'],'classification__C':[0.001,0.01,0.1,1,10]}

grid_search1=GridSearchCV(estimator=model,param_grid=ker_c, scoring='recall',cv=5)


grid_search1.fit(X_train,Y_train)

best_params=grid_search1.best_params_
best_params
best_res= grid_search1.best_score_
#print(best_res)
```

This includes:

- Importing the pipeline from the library
- Smoting the data for balancing the trained data
- Kernel defines the type of decision boundary
- Analysing the best parameters for accuracy evaluation

Accuracy prediction with grid search

The procedure follows by adjusting hyperparameters like C and kernel, then testing the optimized model on the test set in order to show the accuracy of an SVM model with a Grid Search mechanism.

```
[ ] from sklearn import svm # Loading library of svm
    from sklearn.svm import SVC # Loading function SVC from svm library
    svm_classifier_rbf= SVC(kernel='rbf',C=10,random_state=1) # Building classifier
    svm_classifier_rbf.fit(X_train,Y_train) # Building classifier on the train X and Y train values
```

```
SVC
SVC(C=10, random_state=1)
```

This includes:

- Loading the SVM libraries
- Finding SVM classifier using optimized model

Finding accuracy:

```
# Predicting the test values using the classifier
Y_pred=svm_classifier_rbf.predict(X_test)

# Evaluating the model based on chosen kernel
from sklearn import metrics

Accuracy= metrics.accuracy_score(Y_test,Y_pred)
Precision=metrics.precision_score(Y_test, Y_pred)
Recall = metrics.recall_score(Y_test,Y_pred)
F1= metrics.f1_score(Y_test,Y_pred)

print('Accuracy:', Accuracy)
print('Precision:', Precision)
print('Recall:', Recall)
print('F1 score:', F1)
```

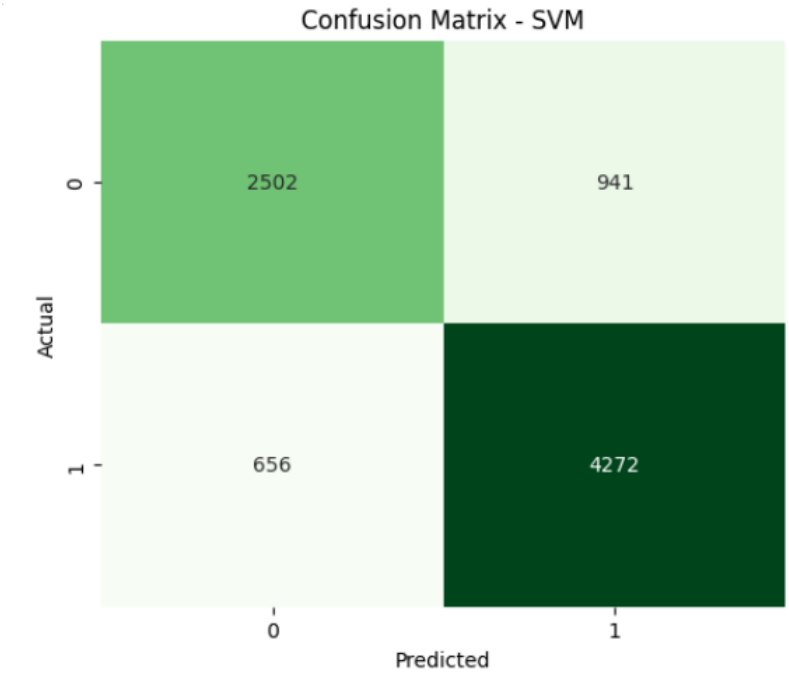
```
Accuracy: 0.8092223151355872
Precision: 0.8194897371954729
Recall: 0.8668831168831169
F1 score: 0.8425204614929495
```

Confusion matrix after optimization

The confusion matrix can be used to assess the model's classification performance following Grid Search-based hyperparameter optimization for the Support Vector Machine (SVM).

```
[ ] conf_mat=metrics.confusion_matrix(Y_test,Y_pred)
    conf_mat
```

```
array([[2502,  941],
       [ 656, 4272]])
```



By optimizing the model with grid search, we have increased the accuracy of the model and thereby ensures relatively higher accurate predictions.

7. Model Comparision

When comparing machine learning models, one should consider several elements, including accuracy, training time, interpretability, resistance to noise and overfitting, and the capability to manage several data sources. Based on their features, utility, and use cases, the Random Forest, SVM, and decision tree models will be compared below.

```
from sklearn.metrics import accuracy_score, log_loss
# List of Classifiers

classifiers = [
    dec_tree_best_depth_impt_feat,
    RF2,
    svm_classifier_rbf
]

```

Calculating the accuracy and log-in loss for every classifier helps us determine which one performs best among them.

```
#Accuracy and log loss of each classifier
log_cols=["Classifier", "Accuracy", "Log Loss"]
log = pd.DataFrame(columns=log_cols) # Changed 'Dataframe' to 'DataFrame'
for clf in classifiers:
  clf.fit( X_train, Y_train)
  name = clf.__class__.__name__
  print("="*30)
  print(name)
  print('***Results***')
#prediction
  train_predictions = clf.predict(X_test)
  acc = accuracy_score(Y_test,train_predictions)
  print("Accuracy: {:.4%}".format(acc))
# log loss require probabilities
  log_entry = pd.DataFrame([[name, acc*100, 11]], columns=log_cols)
  log = pd.concat([log, log_entry], ignore_index=True)
  print("="*30)
  print(log)
```

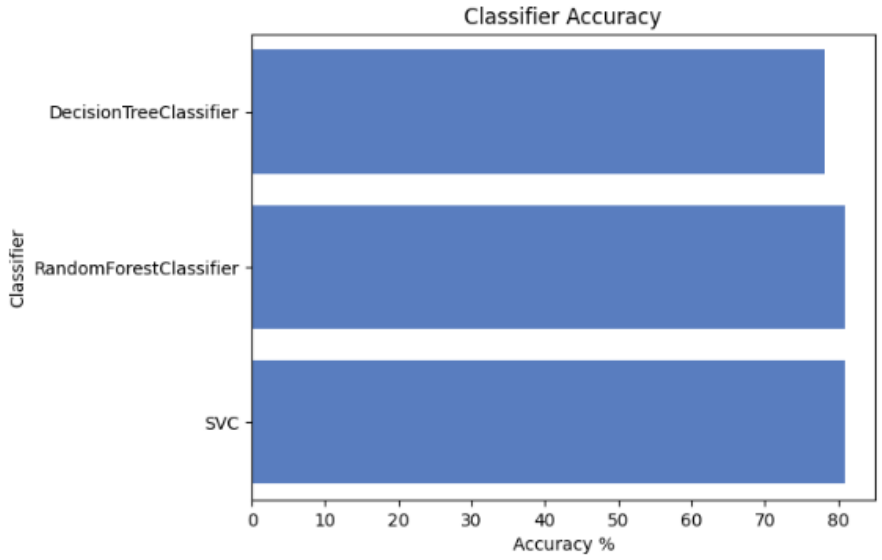
The accuracy of each model can be obtained from the execution of the aforementioned code as follows:

```
==============================
DecisionTreeClassifier
***Results***
Accuracy: 78.1627%
==============================
              Classifier   Accuracy Log Loss
0  DecisionTreeClassifier  78.162705       11
<ipython-input-106-784f70d4fd98>:22: FutureWarning: The be
  log = pd.concat([log, log_entry], ignore_index=True)
==============================
RandomForestClassifier
***Results***
Accuracy: 80.9581%
==============================
              Classifier   Accuracy Log Loss
0  DecisionTreeClassifier  78.162705       11
1  RandomForestClassifier  80.958070       11
==============================
SVC
***Results***
Accuracy: 80.9222%
==============================
              Classifier   Accuracy Log Loss
0  DecisionTreeClassifier  78.162705       11
1  RandomForestClassifier  80.958070       11
2                     SVC  80.922232       11
```

A clearer perspective on these model comparisons can be achieved by constructing a bar chart.

```
# Plot a barchart to visualize the accuracy
sns.set_color_codes("muted")
sns.barplot(x='Accuracy', y='Classifier', data=log, color="b")

plt.xlabel('Accuracy %')
plt.title('Classifier Accuracy')
plt.show()
```



The bar plot comparing the three models shows that the Random Forest Classifier and Support Vector Machine (SVM) generate better results compared to the Decision Tree.

Model	Accuracy	Precision	Recall	Training Time	Key Characteristics
Decision Tree	0.7816	0.7270	0.8812	Fastest	Simple, interpretable
Random Forest	0.8095	0.8668	0.8199	Moderate	Robust, handles overfitting
SVM	0.8092	0.8194	0.8668	Slowest	Good for high-dimensional data

Decision Tree

- ❖ Strengths:

- Fastest training time, making it suitable for quick prototyping or scenarios where speed is critical.

- High recall (0.8812), indicating it is effective at capturing positive cases, which is useful in applications like medical diagnosis or fraud detection where missing positives is costly.

- Simple and interpretable, ideal for explaining model decisions to non-technical stakeholders.

❖ **Weaknesses**:

- Lowest accuracy (0.7816) and precision (0.7270) among the three models, suggesting it may make more false positives compared to the others.

- Prone to overfitting, especially with complex trees, though this isn't explicitly reflected in the table.

Random Forest

❖ **Strengths**:

- Highest accuracy (0.8095) and precision (0.8668), indicating better overall performance and fewer false positives compared to the Decision Tree.

- Robust and handles overfitting well due to its ensemble nature (multiple trees averaging results).

- Balanced recall (0.8199), making it a good all-rounder for many applications.

❖ **Weaknesses**:

- Moderate training time, which may be slower than a single Decision Tree but still reasonable for most use cases.

- Less interpretable than a single Decision Tree due to the ensemble approach.

SVM (Support Vector Machine)

❖ **Strengths:**

- High recall (0.8668) and competitive accuracy (0.8092), similar to Random Forest but with a slight edge in recall.

- Precision (0.8194) is decent, though not as high as Random Forest.

- Performs well with high-dimensional data, making it suitable for tasks like text classification or image recognition.

❖ **Weaknesses**:

- Slowest training time (noted as "Ctrl - Slowest"), which could be a bottleneck for large datasets or real-time applications.

- Less interpretable compared to tree-based models, with more complex parameter tuning (e.g., kernel selection).

Overall Comparison:

- Best for Accuracy/Precision: Random Forest (highest in both metrics).

- Best for Recall: Decision Tree (highest recall, but at the cost of precision).

- Best for Interpretability: Decision Tree (simple and transparent).

- Best for High-Dimensional Data: SVM (as highlighted in its key characteristics).

- Best Training Speed: Decision Tree (fastest), followed by Random Forest, with SVM being the slowest.

Recommendation:

- Choose Random Forest if you prioritize balanced performance (accuracy, precision, and recall) and robustness.

- Opt for Decision Tree if speed and interpretability are critical, and you can tolerate slightly lower accuracy/precision.

- Use SVM if you're working with high-dimensional data and need strong recall, but be mindful of its slower training time.

8. Individual contributions

Name	**Contribution**
Nguyen Van Ly	I am the leader of this project, and as such, I am responsible for assigning tasks, reviewing code, assisting other members in the development of models, fixing issues, and consolidating reports. Within the report, I was responsible for the selection of algorithms, the selection of features, the selection of data, and the comparison of models. I also chose the dependent and independent variables. Additionally, Besies, I, and Swathy collaborate on the development of Random Forest, and we also assist Angelin in the optimization of the Decision Tree model. Following the completion of the mode by all participants, I review and compose a conclusion.
Angelin Robinson	I played a key role in identifying and sourcing the dataset from various websites. After obtaining the dataset, I modified and refined it to ensure its suitability for analysis. Additionally, I wrote the Introduction and Data Description sections of the report.

	In terms of analysis, I conducted data understanding and preprocessing, including selection of dependent and independent variables and feature selection to optimize model performance. I developed and implemented the Decision Tree algorithm, ensuring proper model training, testing, and evaluation. Furthermore, I documented and analyzed the Decision Tree results, providing a comprehensive report on its performance and insights derived from the model. I wrote a report on the decision tree Algorithm. My contributions covered multiple critical aspects of the project, from data acquisition and preparation to model development, evaluation, and reporting
Swathy Panampilly Saseendran	I led the development and implementation of the Random Forest classification model for this project. My work encompassed the full machine learning pipeline - from writing and executing the model code to conducting comprehensive performance evaluations. I specialized in hyperparameter optimization to enhance the model's predictive capabilities, using methodical testing to determine the most effective configuration. Through detailed analysis of confusion matrices and classification metrics, I validated the model's suitability for mental health screening applications. Finally, I documented the entire process in a technical report that clearly explains the methodology, results, and practical recommendations for implementation.
Alfas Jasim	Played a major role in accessing the dataset and contributed in the process of refining and cleaning the dataset to the preferred qualities for the models to work efficiently. Done my work on the SVM(Support Vector Machine) for this project and predicted the outcomes based on the SVM algorithm without using grid search and with using grid search for optimizing the model to get the best results, also applied confusion matrix and derived significant insights from the model to predict the values required for finding the student depression. Finally prepared the report for my part in the project by giving significant informations of the model that I implemented.